Funciones, Variables Globales y Locales, y Módulos

Estructura del programa · Cierre del Capítulo 2

Carlos Pinilla

pinilla.carlos@correounivalle.edu.co

Contenido

1. Procedimientos: por qué dividir un programa.
2. Funciones y subrutinas.
3. Paso de argumentos e `intent`.
4. Variables locales y globales.
5. Módulos.
6. Recapitulación del Capítulo 2.
7. Taller 6: fórmula semiempírica de masa.

Continuidad con las semanas anteriores

Semanas 1 a 5

- Un solo bloque: datos, cálculo y salida en el mismo `program`.
- Un cálculo repetido se copia: tres casos de verificación, veinte corridas de Monte Carlo.

Semana 6

- El cálculo se escribe **una vez**, con nombre, y se llama muchas veces.
- Constantes y procedimientos se agrupan en módulos.

Cada concepto se presenta en Fortran y en Python.

Motivación: el programa del Taller 1

$$h_{\text{máx}} = \frac{(v_0 \sin\theta)^2}{2g}$$

v_0: rapidez inicial [m/s]; θ: ángulo con la horizontal; $g = 9.8$ m/s^2.

v_0 [m/s]	θ [°]	$h_{\text{máx}}$ [m]
9.8	90	4.9000
20	45	10.2041
20	30	5.1020

- Verificar tres casos exigía **ejecutar tres veces** el programa.
- La unidad que se podía repetir era el programa completo.

Procedimiento

Bloque de código con nombre que recibe datos por **argumentos**, hace un cálculo y entrega un resultado. Se escribe una vez y se llama las veces que haga falta.

Función en Fortran

```
program funcion_altura
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  ! la función se usa dentro de una expresión
  print *, altura_maxima(20.0_dp, 45.0_dp)
contains            ! aquí empiezan los procedimientos
  real(dp) function altura_maxima(v0, theta_grados)
    real(dp), intent(in) :: v0, theta_grados
    real(dp), parameter :: g = 9.8_dp
    real(dp), parameter :: pi = 4*atan(1.0_dp)
    real(dp) :: theta                  ! local
    theta = theta_grados*pi/180.0_dp
    ! el resultado se asigna al nombre
    altura_maxima = (v0*sin(theta))**2/(2*g)
  end function altura_maxima
end program funcion_altura
```

- Va después de `contains`.
- El encabezado declara el **tipo del resultado**.
- El resultado se asigna al **nombre** de la función.

Salida: 10.2041 m, el segundo caso de la tabla.

Función en Python

```python
import math

def altura_maxima(v0, theta_grados):
    """Entradas: v0 [m/s], theta [grados].
    Salida: altura máxima [m]."""
    g = 9.8                                  # local
    theta = math.radians(theta_grados)
    return (v0*math.sin(theta))**2/(2*g)

for v0, th in [(9.8, 90), (20, 45), (20, 30)]:
    print(altura_maxima(v0, th))
```

- `def` define la función; `return` entrega el valor.
- Sin `return`, la función devuelve `None`.

Resultado

Los dos lenguajes dan 4.9000, 10.2041 y 5.1020: la tabla del Taller 1, con una sola fórmula.

Subrutina en Fortran: varios resultados

$$h_{\text{máx}} = \frac{v_0^2 \sin^2\theta}{2g}, \qquad R = \frac{v_0^2 \sin 2\theta}{g}, \qquad t_v = \frac{2v_0 \sin\theta}{g}$$

```
! los resultados llegan por los argumentos h, r, t
call tiro_parabolico(20.0_dp, 45.0_dp, h, r, t)
...
subroutine tiro_parabolico(v0, theta_grados, &
                           h_max, alcance, t_vuelo)
  real(dp), intent(in)  :: v0, theta_grados
  real(dp), intent(out) :: h_max, alcance, t_vuelo
  ...
end subroutine tiro_parabolico
```

$h_{\text{máx}}$	10.2041 m
R	40.8163 m
t_v	2.8862 s

En Python: `return h, r, t` devuelve una tupla, y `h, r, t = tiro_parabolico(20, 45)` la reparte.

Criterio de elección: función o subrutina

	Función	**Subrutina**
Cómo se invoca	En una expresión: `y = f(x)`	Como instrucción: `call s(x, y, z)`
Qué entrega	Un valor, por su nombre	Resultados por sus argumentos
Sus argumentos	Todos `intent(in)`	`in`, `out` o `inout`
Intrínsecas conocidas	`sin`, `sqrt`, `abs`	`random_number`, `random_seed`

Regla práctica

1. Un solo resultado y nada más: **función**.
2. Varios resultados: **subrutina**.
3. Una acción (imprimir, escribir un archivo): **subrutina**.

Paso de argumentos: Fortran y Python

Fortran: por referencia

```
angulo = 30.0_dp
call imprimir_seno(angulo)
print *, angulo      ! 0.523599, ya no 30
...
subroutine imprimir_seno(theta)
  real(dp) :: theta  ! sin intent
  theta = theta*pi/180.0_dp
```

El argumento ficticio **es la misma variable** del programa que llama.

Python: el número no cambia

```
def imprimir_seno(theta):
    theta = math.radians(theta)
    print(math.sin(theta))

angulo = 30.0
imprimir_seno(angulo)
print(angulo)        # 30.0
```

Reasignar el parámetro crea un objeto nuevo; el del llamador no cambia.

El atributo `intent`

`intent`	Significado	Si se viola
`in`	El argumento solo entra	Error de compilación
`out`	El argumento solo sale	Se pierde el valor de entrada
`inout`	Entra y sale modificado	—

Con `intent(in)`, el ejemplo anterior ya no compila:

```
Error: Dummy argument 'theta' with INTENT(IN) in variable definition context
```

Regla del curso

Todos los argumentos de todos los procedimientos llevan `intent`: un resultado equivocado se convierte en un error de compilación.

Variables globales en Python

```
G = 9.8                    # global
def peso(m):
    return m*G             # solo lee G: correcto

contador = 0
def contar_llamada():
    contador = contador + 1    # asigna: es LOCAL
    return contador
# UnboundLocalError: cannot access local
# variable 'contador'
```

- Si una función **asigna** un nombre, ese nombre es local en toda la función.
- `global contador` indica que se refiere a la global.

Regla del curso

Globales solo para **constantes** (g, π, coeficientes). Los resultados se devuelven con `return`.

Asociación por anfitrión en Fortran

```
k = 1
do while (k <= 3)
   f = factorial(4)
   print *, 'caso', k, ':  4! =', f
   k = k + 1
end do
contains
integer function factorial(n)
  integer, intent(in) :: n
  factorial = 1
  do k = 2, n          ! k NO declarada aquí
     factorial = factorial*k
  end do
```

Salida: `caso 5: 4! = 24`

- Un procedimiento interno ve las variables del programa que lo contiene.
- `factorial` usó el `k` del programa y lo dejó en 5: el ciclo terminó en la primera vuelta.
- Compila sin advertencias, aun con `implicit none`.

Corrección

Declarar `integer :: k` dentro de la función.

Inicialización en la declaración: save implícito

```
subroutine con_save()
  integer :: llamadas = 0    ! UNA sola vez
  llamadas = llamadas + 1
  print *, llamadas
end subroutine con_save

subroutine sin_save()
  integer :: llamadas
  llamadas = 0               ! en CADA llamada
  llamadas = llamadas + 1
  print *, llamadas
end subroutine sin_save
```

Tres llamadas a cada una:

	1.ª	2.ª	3.ª
con_save	1	2	3
sin_save	1	1	1

Consecuencia

Un acumulador inicializado en su declaración empieza, desde la segunda llamada, con el valor de la anterior. En Python esto no ocurre.

Módulos en Fortran

```
module mod_proyectil          ! mod_proyectil.f90
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  real(dp), parameter :: g = 9.8_dp    ! constante
contains                       ! procedimientos
  real(dp) function alcance(v0, theta_grados)
    ...
end module mod_proyectil
```

```
program usa_modulo            ! usa_modulo.f90
  use mod_proyectil, only: dp, alcance
  implicit none
  print *, alcance(20.0_dp, 45.0_dp)    ! 40.8163
end program usa_modulo
```

- Agrupa constantes y procedimientos en un archivo propio.
- `use` va antes de `implicit none`; `only:` importa solo lo nombrado.

Orden de compilación

Compilar el módulo genera `mod_proyectil.mod`, que necesita el programa. **El módulo se compila primero**; si no:

```
Cannot open module file
'mod_proyectil.mod'
```

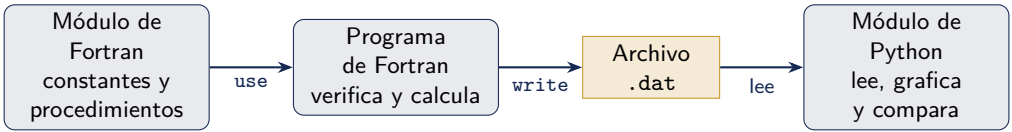
Módulos en Python

```
%%writefile proyectil.py
import math
G = 9.8                # constante del módulo
def alcance(v0, theta_grados):
    theta = math.radians(theta_grados)
    return v0**2*math.sin(2*theta)/G
```

```
import importlib, proyectil
importlib.reload(proyectil)   # relee si cambió
print(proyectil.alcance(20.0, 45.0))   # 40.8163
```

- Todo archivo `.py` es un módulo.
- Se importa con `import`; sus nombres se usan con el prefijo del módulo.
- Python importa un módulo una sola vez por sesión: `importlib.reload` obliga a leerlo de nuevo.

Flujo de trabajo: Fortran calcula, Python analiza



- Ejemplo integrador del notebook: los tres casos del Taller 1 dan PASS con tres llamadas, y Fortran escribe una tabla de $h_{\text{máx}}$ y R para $\theta = 5^\circ, \ldots, 85^\circ$.
- Python recalcula la tabla con su propio módulo: la diferencia máxima es 5×10^{-9} m, el redondeo de los 8 decimales del archivo.

Es el mismo flujo de las Semanas 4 y 5, y el del Taller 6.

Recapitulación del Capítulo 2

Lo que pide el Taller 6	**Dónde se vio**
Completar el pseudocódigo antes de programar	Semana 1
`real(dp)` y evitar la división entera en $A^{-1/3}$	Semana 2
Ciclos anidados y un condicional que guarda el mínimo	Semanas 2 a 5 ($\mathcal{C}^3$)
Comparar un modelo con datos experimentales	Semanas 2 y 3
Fortran escribe `.dat`; Python los lee y grafica	Semanas 4 y 5
Verificación con casos conocidos (PASS/FAIL)	Todas las semanas
Funciones, subrutinas, `intent`, módulos	Semana 6

Taller 6: fórmula semiempírica de masa

Energía de enlace por nucleón de un núcleo con Z protones y A nucleones (modelo de la gota líquida):

$$E_B(Z,A) = \underbrace{-\,a_V}_{\text{volumen}} + \underbrace{a_S\,A^{-1/3}}_{\text{superficie}} + \underbrace{a_C\,Z^2A^{-4/3}}_{\text{Coulomb}} + \underbrace{a_A\,\frac{(Z-A/2)^2}{A^2}}_{\text{asimetría}}$$

- **Volumen**: la fuerza nuclear es de corto alcance; cada nucleón liga con sus vecinos.
- **Superficie**: los nucleones del borde tienen menos vecinos ($R \propto A^{1/3}$).
- **Coulomb**: repulsión entre los Z protones.
- **Asimetría**: el principio de Pauli favorece $Z = N = A/2$.

Coeficientes de O. Zubairi y F. Weber (2024), problema 7.2. Convención: $E_B < 0$ para un núcleo ligado.

Taller 6: preguntas y entregables

El programa responde

1. Los cuatro términos, E_B y M de ocho núcleos.
2. Para cada A, el Z más estable: el **valle de estabilidad**.
3. El núcleo más ligado de todo el valle.

Verificación (4 casos): ^{244}Pu contra el libro, $Z = 82$ para $A = 208$, $Z = 92$ para $A = 238$, y el mismo máximo en Fortran y en Python.

Se entrega

- El pseudocódigo marco, con sus tres espacios completos.
- Fortran: `modulo_nucleo.f90` y `programa_nucleo.f90`.
- Python: `analisis_nucleo.py` y la curva B/A contra A.
- Respuestas de interpretación y reporte de una página.

Sin esqueleto de código: la organización en procedimientos es parte de lo que se evalúa, y cada estudiante sustentará su programa.

Ejercicios de calentamiento para el lunes

1. **¿Función o subrutina?** Tiempo de vuelo: obtener $t_v = 2.8862$ s para $v_0 = 20$ m/s y $\theta = 45^\circ$, y justificar la elección.

2. **Errores de ámbito.** Un programa que compila sin advertencias imprime dos líneas en lugar de tres: encontrar y corregir sus dos errores.

3. **El Taller en pequeño.** Llevar el tiempo de vuelo a un módulo en Fortran y en Python, y obtener el mismo valor en los dos.

Están en el notebook, en recuadros dorados. No se califican; preparan el Taller 6.

Conclusiones

- Una **función** entrega un valor y se usa en una expresión; una **subrutina** entrega varios resultados o realiza una acción.
- En Fortran los argumentos son la misma variable del llamador: `intent` los protege.
- Las globales se reservan para constantes; cada procedimiento declara sus variables de trabajo.
- Un módulo agrupa constantes y procedimientos, y se compila primero.

Próxima semana: arreglos (Capítulo 3).

Referencias

- S. J. Chapman, *Fortran for Scientists and Engineers*, 4.ª ed., McGraw-Hill, 2018.
- O. Zubairi y F. Weber, *Introduction to Computational Physics for Undergraduates*, 2.ª ed., IOP Publishing, 2024, sec. 3.8 y problema 7.2.
- R. H. Landau, M. J. Páez y C. C. Bordeianu, *Computational Physics: Problem Solving with Python*, 4.ª ed., Wiley.